**INGENIERÍA DE SEGURIDAD US**
**E.T.S. Ingeniería Informática. Universidad de Sevilla**
Avda. Reina Mercedes s/n. 41012 Sevilla
**Tlf/Fax** 954 557 139 **E-mail** lsi@lsi.us.es **Web** www.lsi.us.es

INSEGUS

# PAI-3. VULNAWEB. AUDITORÍA DE SEGURIDAD EN SISTEMAS INFORMÁTICOS Y ANÁLISIS DE VULNERABILIDADES EN UNA UNIVERSIDAD PÚBLICA

## Introducción

De acuerdo con el Plan Director de Seguridad de la Información, la organización ha avanzado en diversos proyectos orientados a garantizar la seguridad en la transmisión de información a través de redes públicas (cableadas e inalámbricas). Dando continuidad a esta estrategia, el presente proyecto actualiza el marco de auditoría de seguridad de la Universidad Pública U-Secure en colaboración con INSEGUS.

Esta evolución supone la transición desde un análisis clásico hacia un modelo de postura de seguridad integral (Endpoints + DAST), garantizando el cumplimiento y alineamiento con los estándares como OWASP Top 10, CIS Benchmarks, NIST SP 800-115 y el Esquema Nacional de Seguridad (ENS - Categoría Alta).



Figura 1: Ejemplo de sistemas informáticos

El alcance del proyecto abarca la auditoría técnica, evaluación de vulnerabilidades y definición de planes de remediación sobre los sistemas informáticos y aplicaciones que prestan servicio a la comunidad universitaria. La infraestructura para auditar comprende dos bloques fundamentales:

1. **Endpoints de la Comunidad Universitaria** (Equipos Corporativos y BYOD):
    - **Equipos de Sobremesa y Portátiles (Linux)**: Auditoría de bastionado (hardening) del sistema operativo y fortificación de navegadores web utilizados por el Personal Docente e Investigador (PDI) y el Personal Técnico, de Gestión y de Administración y Servicios (PTGAS).
    - **Dispositivos Móviles (Android/iOS)**: Evaluación de la postura de seguridad en dispositivos BYOD, control de permisos, ausencia de elevación de privilegios (rooting) y mitigación de riesgos en el acceso a servicios institucionales.
2. **Plataformas y Servicios Web Institucionales** (DAST / Pentesting Web):

CONFIDENCIAL

- **Entornos Evaluables**: Aplicaciones y servicios web en producción y entornos de pruebas que gestionan trámites administrativos, pasarelas de pago y datos sensibles.
- **Vectores de Prueba**: Intercepción de tráfico cifrado HTTPS mediante proxy interceptor (OWASP ZAP / Burp Suite), fuzzing automatizado de parámetros, análisis de trazabilidad/cabeceras de seguridad y pruebas de penetración enfocadas en vectores críticos del OWASP Top 10 (SQL Injection, Cross-Site Scripting y Path Traversal).

NOTA MUY IMPORTANTE: El mal uso de estas herramientas puede llevar a situaciones de pérdida de la disponibilidad, confidencialidad e integridad de la información, bloqueo, lentitud en la velocidad de conexión e incluso denegación de servicio de las máquinas analizadas e incluso problemas a los usuarios del servicio. Estas herramientas no deben ser utilizadas contra servidores en producción para los que no se encuentre debidamente autorizado. El único fin de este proyecto es didáctico, con el objeto de dar a conocer cómo realizar auditorías de seguridad informática de sistemas informáticos y aplicaciones Web, y se debe evitar el uso de dichas herramientas para otros fines ilegales o no lícitos.

## Objetivos

Por tanto, los objetivos de este proyecto son los siguientes:

- **Auditoría y Bastionado (Hardening) de Endpoints**: Auditoría de seguridad de los endpoints (sistemas informáticos de sobremesa, portátiles y dispositivos móviles) que utilizan los usuarios de los usuarios PDI, Estudiantes, y PTGAS de la Universidad para determinar su postura de seguridad.
- **Intercepción y Trazabilidad de Tráfico Cifrado HTTPS**: Desplegar y configurar un proxy interceptor (OWASP ZAP / Burp Suite) demonstrando la trazabilidad de tráfico http/https a través de dicho proxy.
- **Pentesting Web y Fuzzing Automatizado (OWASP Top 10)**: Auditoría de las aplicaciones Web que dan soporte a las actividades de la universidad pública, identificando posibles vulnerabilidades de seguridad, tales como Inyecciones, XSS, CSRF, Path Traversal, etc.
- **Generación de Evidencias Técnicas**: realizar las pruebas y capturar las evidencias de las auditorías realizadas.

CONFIDENCIAL

## Análisis de sistemas informáticos (sobremesa y/o portátil)

La herramienta Lynis que permite realizar auditorías bastante completas sobre seguridad de servidores y sistemas informáticos con sistemas operativos Linux o Unix. Esta herramienta está disponoble con las distribuciones Linux más conocidas y usadas como Ubuntu, Arch, Debian, Fedora y OpenSUSE.

Lynis determina la postura de seguridad usando una métrica denominada índice de hardening. Lynis escanea el sistema informático revisando el software instalado y su configuración, determinando el cumplimiento con los estándares y las buenas prácticas en seguridad de la información.

Una vez instalada la herramienta podemos probar un escaneo rápido desde la terminal con el siguiente comando:

> sudo lynis –Q[1]

Si se desea un auditoria más completa se puede usar check-all (o -c). Lynis analizará el gestor de arranque del sistema auditado, los servicios que arrancamos al iniciar y diferentes aspectos sobre el kernel. Además, veremos, entre otras cosas, información sobre:

- Herramientas del Sistema
- Procesos y memoria
- Usuarios, grupos y métodos de autenticación
- Shells
- Sistema de Archivos
- Almacenamiento
- Sistemas de Archivos en la Red, NFS
- Aplicaciones

Este conjunto de comprobaciones para analizar la seguridad del sistema informático se muestra al final del análisis. Cada resultado debe interpretarse por el correspondiente auditor y comprobar que significa. La salida de las pruebas puede ser [OK] o [WARNING], donde el primero es considerado como un resultado ESPERADO, y el segundo es un resultado NO ESPERADO (de acuerdo con la política, no quiere decir que sea incorrecto). También puede aparecer [NO FOUND], como que no se ha encontrado el objeto de testeo. Además, información adicional sobre las comprobaciones realizadas se guardan en un fichero de logs que por defecto es /var/log/lynis.log, que es actualizado en cada ejecución. Esta información será muy útil para ver que hace la herramienta en background o dónde las anomalías aparecen (y frecuentemente porqué).

[1] La Q indica análisis rápido

CONFIDENCIAL

El informe presenta un conjunto de sugerencias tales como se ve a continuación antes de mostrar el hardening index:

- [02:33:38] Suggestion: update to the latest stable release.
- [02:33:41] Suggestion: Install a PAM module for password strength testing like pam_cracklib or pam_passwdqc [test:AUTH-9262]
- [02:33:41] Suggestion: When possible set expire dates for all password protected accounts [test:AUTH-9282]
- [02:33:41] Suggestion: Configure password aging limits to enforce password changing on a regular base [test:AUTH-9286]
- [02:33:41] Suggestion: Default umask in /etc/profile could be more strict like 027 [test:AUTH-9328]
- [02:33:41] Suggestion: Default umask in /etc/login.defs could be more strict like 027 [test:AUTH-9328]
- [02:33:41] Suggestion: Default umask in /etc/init.d/rc could be more strict like 027 [test:AUTH-9328]
- [02:33:41] Suggestion: To decrease the impact of a full /tmp file system, place /tmp on a separated partition [test:FILE-6310]
- [02:33:42] Suggestion: Disable drivers like USB storage when not used, to prevent unauthorized storage or data theft [test:STRG-1840]
- [02:33:42] Suggestion: Disable drivers like firewire storage when not used, to prevent unauthorized storage or data theft [test:STRG-1846]
- [02:33:54] Suggestion: Install package apt-show-versions for patch management purposes [test:PKGS-7394]
- [02:33:55] Suggestion: Configure a firewall/packet filter to filter incoming and outgoing traffic [test:FIRE-4590]
- [02:33:56] Suggestion: Add legal banner to /etc/issue, to warn unauthorized users [test:BANN-7126]
- [02:33:56] Suggestion: Add legal banner to /etc/issue.net, to warn unauthorized users [test:BANN-7130]

Posteriormente, Lynis presenta los nombres de los ficheros con los informes y el hardening index calculado:

```
Files:
  - Test and debug information      : /var/log/lynis.log
  - Report data                     : /var/log/lynis-report.dat
  - Current version: 129    Latest version : 139
    ============================================================
  - Hardening index : [44]     [########  ]
```

Se observa que el hardening index es 44 y, por ejemplo, la Política de Seguridad de la empresa nos indicara que deben tener un Hardening Index superior, para ello se plantean las siguientes acciones:

- Realizar las acciones que se indican en las sugerencias de la herramienta para mejorar el Hardening Index.
- Realizar acciones teniendo en cuenta los WARNING:
- Resolver el problema: Leyendo el fichero de logs acerca de los detalles técnicos.
- Deshabilitar la prueba no de interés de la lista: Dentro del perfil de escaneado, los tests se pueden deshabilitar porque no estés interesado en ello, con la opción= test_skip_always.

CONFIDENCIAL

El objetivo sería **realizar una auditoría de tres/dos sistemas informáticos**. Realizar la correspondiente auditoría y **tomar todas las medidas necesarias para alcanzar el hardening index superior o igual a 70** según lo especificado en la Política de Seguridad de la empresa. En caso de no alcanzar este hardening index **se deberá realizar todas las actuaciones que sean oportunas para bastionar estos sistemas**. Todas las actuaciones realizadas para mejorar el hardening index deberá ser documentadas en el informe final de este proyecto, indicando claramente y de manera justificada las razones por las cuales ha seleccionado unas determinadas acciones y no otras.

### Análisis de navegadores web

Los navegadores web (Mozilla Firefox, Opera, Edge, Chrome, Brave, Chromiun, etc.) son aplicaciones críticas ya que permiten la conexión de los usuarios de la universidad con los servicios corporativos y pueden ser utilizados como vector de ataque. Por lo tanto, se deben garantizar la postura de seguridad de dichas aplicaciones.

El objetivo de esta actuación es analizar y fortificar los navegadores web presentes en los sistemas informáticos de la organización. Para lograrlo, se definirá e implementará un conjunto de directivas corporativas de bastionado (browser hardening) orientadas a establecer un perfil de configuración seguro y resistente frente a ciberamenazas. El informe del proyecto incluirá la justificación detallada y trazable de cada una de las directivas y ajustes técnicos aplicados en dichos navegadores.

### Análisis de un sistema móvil

Cada equipo de evaluación seleccionará la herramienta de análisis de seguridad móvil que considere más conveniente para auditorías en entornos Android/iOS (tales como MobSF - Mobile Security Framework, Koodous Mobile o Malwarebytes).

El objetivo prioritario es determinar la postura de seguridad del dispositivo BYOD del usuario (PDI / PTGAS) y ejecutar las acciones correctoras necesarias para alcanzar el nivel de bastionado (hardening) exigido por la Política de Seguridad de la Universidad Pública.

CONFIDENCIAL

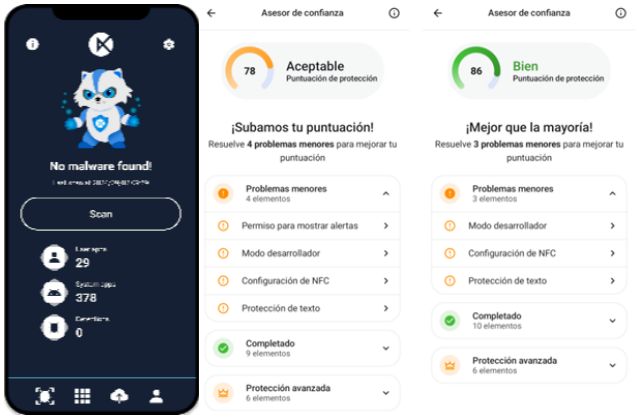



Figura 2: Análisis de dispositivo móvil con Koodous y Malwarebytes

El objetivo sería que cada Security Team analizara **al menos dos dispositivos móviles** con alguna aplicación de análisis de configuración de seguridad y realizara las acciones necesarias para **alcanzar un alto nivel de hardening**. Para alcanzar un nivel alto de hardening, la Política de Seguridad sugiere las siguientes medidas:

## A. Configuración de Seguridad del Dispositivo y Conectividad

- **Orígenes Desconocidos:** Desactivación estricta de la instalación de aplicaciones desde fuentes no oficiales o fuera de Google Play Store (Unknown Sources / Sideloading deshabilitado).
- **Integridad del Entorno (Anti-Rooting/Jailbreak):** Verificación de ausencia de superusuario (rooting) o modificaciones de la imagen del kernel que anulen el aislamiento por sandbox del sistema operativo.
- **Gestión de Actualizaciones**: Comprobación de que el sistema operativo y el navegador web corporativo disponen del último parche de seguridad emitido por el fabricante.
- **Seguridad en Redes Inalámbricas**: Prohibición de conexión a redes Wi-Fi públicas sin cifrado (abiertas), con cifrado obsoleto (WEP/WPA-Basic) o con SSID oculto sin autenticación Enterprise.
- **Conectividad Inalámbrica Restringida**: Desactivación de la interfaz Bluetooth durante las sesiones de acceso a servicios corporativos y Sede Electrónica para mitigar vectores de intercepción de proximidad.

## B. Control de Aplicaciones, Permisos y Tráfico de red

- **Análisis de Malware y Potencialmente Maliciosos**: Verificación mediante escaneo antimalware de que no existen aplicaciones maliciosas, spyware o APKs sospechosas instaladas en el dispositivo.
- **Supervisión de Conexiones en Segundo Plano**: Inspección de conexiones de red (sockets) activos para garantizar que ninguna aplicación no autorizada mantenga conexiones abiertas concurrentes durante el uso del navegador web corporativo.

CONFIDENCIAL

- **Aislamiento de Datos y Permisos Críticos**: Revocación de permisos peligrosos (acceso a almacenamiento compartido, accesibilidad, superposición de pantalla o lectura de notificaciones) a aplicaciones de terceros que puedan interceptar, leer o manipular el tráfico de subida/descarga generada por el navegador web.

## Pentesting Web y Fuzzing Automatizado

Cada Security Team, puede utilizar la herramienta que estime más oportuna para auditar las vulnerabilidades de las aplicaciones web y realizar las acciones correspondientes para alcanzar el hardening requerido por la Política de Seguridad de dicha organización y estándares de buenas prácticas en seguridad Web.

Por si el Security Team no tiene prevista alguna herramienta concreta, les proponemos desde INSEGUS utilizar alguna de las herramientas Dynamic Application Security Testing (DAST) que usan la metodología de testing de seguridad de caja-negra en el que la aplicación Web es auditada desde fuera, como si fuera un posible atacante. Estas herramientas complementan a las de Static Application Security Testing (SAST), que usan una metodología de testing de caja-blanca, donde el auditor examina la aplicación Web desde dentro, buscando en el código fuente los aspectos que expliquen la presencia de ciertas vulnerabilidades de seguridad.

En concreto, una herramienta para llevar a cabo DAST sería la herramienta OWASP ZAP, que es un marco de trabajo para la trazabilidad de peticiones/respuestas HTTP/HTTPS y analizar posibles vulnerabilidades de las aplicaciones web que se comunican con los clientes usando estos protocolos. Su uso más común es operar como un proxy de intercepción, que permite al operador revisar y modificar las peticiones creadas por el navegador antes de que sean enviadas al servidor, y para revisar y modificar respuestas enviadas por el servidor antes de que sean recibidas por el navegador.

CONFIDENCIAL

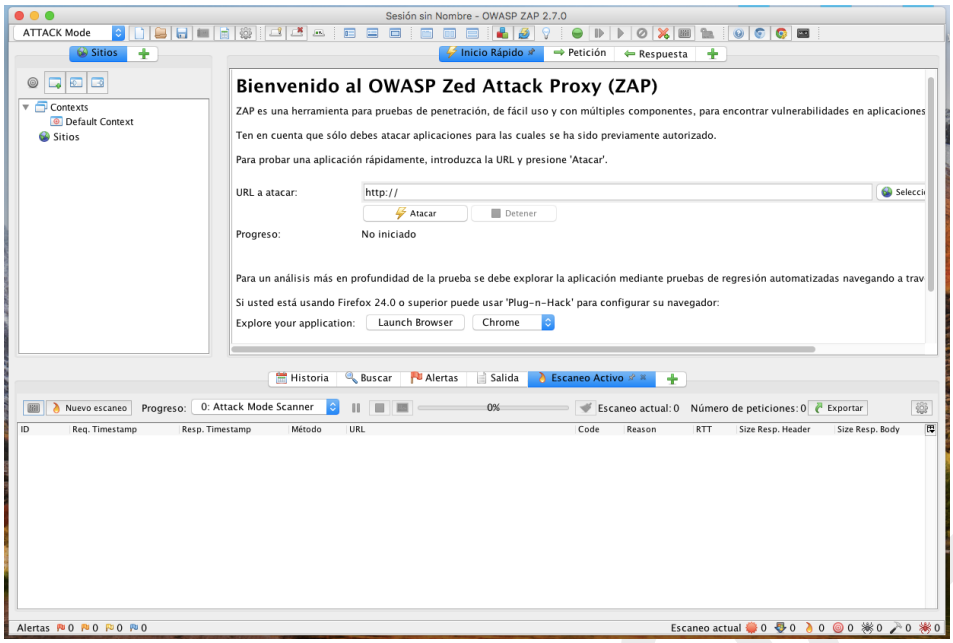



Figura 3: Interfaz de OWASP ZAP

## Configuración de la herramienta OWASP ZAP y del navegador web

En primer lugar, para poder interceptar el tráfico (peticiones/respuestas) hacia una aplicación web vamos a configurar la herramienta para que actúe de proxy. En la herramienta en la pestaña "Proxy" tendremos que configurar los siguientes aspectos:

1. OWASP ZAP: Preferencias
    - Address: dirección IP de escucha
    - Port: puerto de escucha.

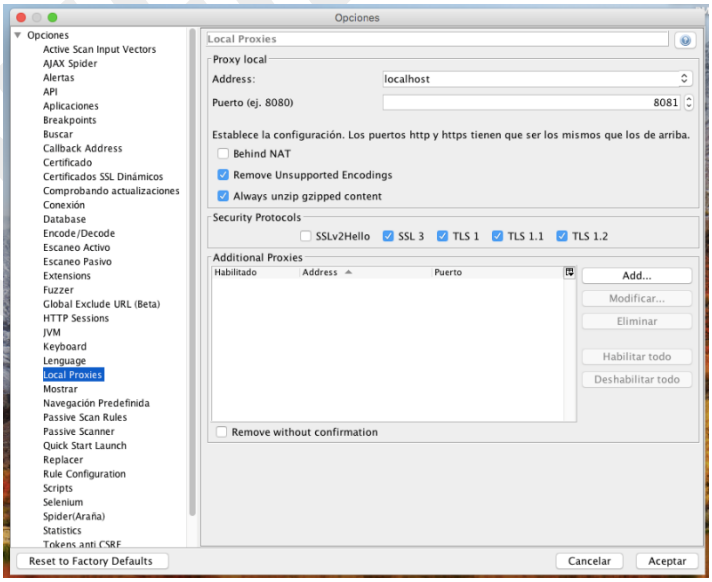



Figura 4: Configuración del proxy en OWASP ZAP

Con estas opciones ya tenemos preparada la herramienta para escuchar todo el tráfico en la dirección/puerto indicadas. Toda petición saliente será interceptada por nuestra herramienta pudiendo así interactuar con las peticiones.

Para comprobar que nuestro proxy está configurado correctamente, podemos lanzar una instancia de un navegador web y usar la dirección localhost:8081 donde está escuchando nuestro proxy.

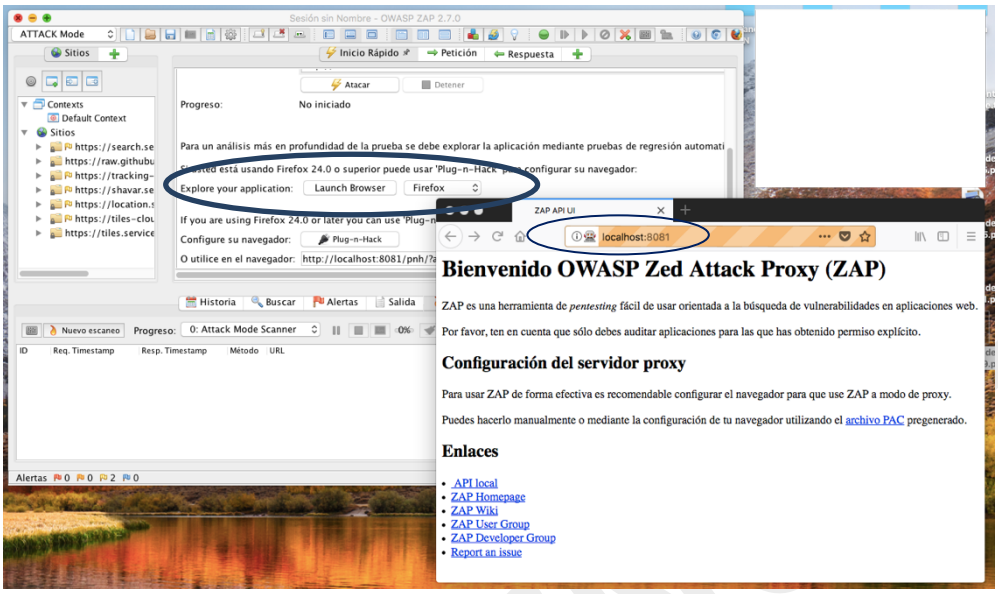



Figura 5: Proxy ZAP funcionando

Podemos usar esta instancia de navegador para realizar las pruebas o configurar nuestro navegador habitual para realizar las pruebas, para ello será necesario configurarlo adecuadamente.

## Configuración manual del navegador web

Una vez configurada la herramienta tendremos la posibilidad también que configurar nuestro navegador web para que todas las peticiones desde nuestro navegador se redirijan al proxy de OWASP ZAP. Mostraremos el ejemplo para un navegador Mozilla Firefox; sin embargo, para otros tipos de navegadores la configuración es bastante parecida.

Paso 1: En las Preferencias del Navegador → General → Network Settings

CONFIDENCIAL

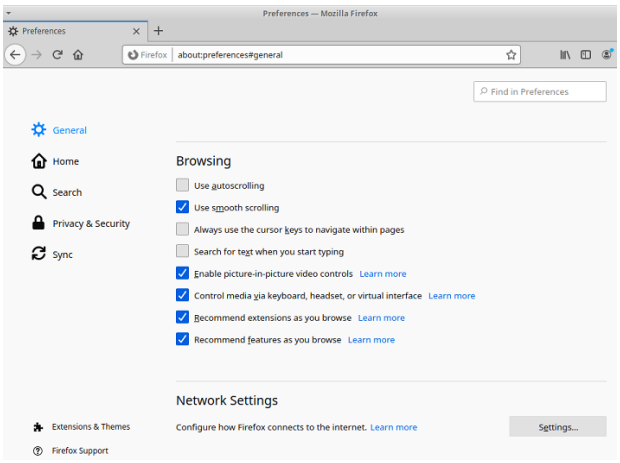



Figura 6: Configuración del navegador

Paso 2: Marcamos la opción de usar “Manual proxy configuration” y “Also use this proxy for FTP and HTTPS”. En HTTP Proxy introduciremos los datos de la dirección IP/puerto que hayamos configurado en ZAP

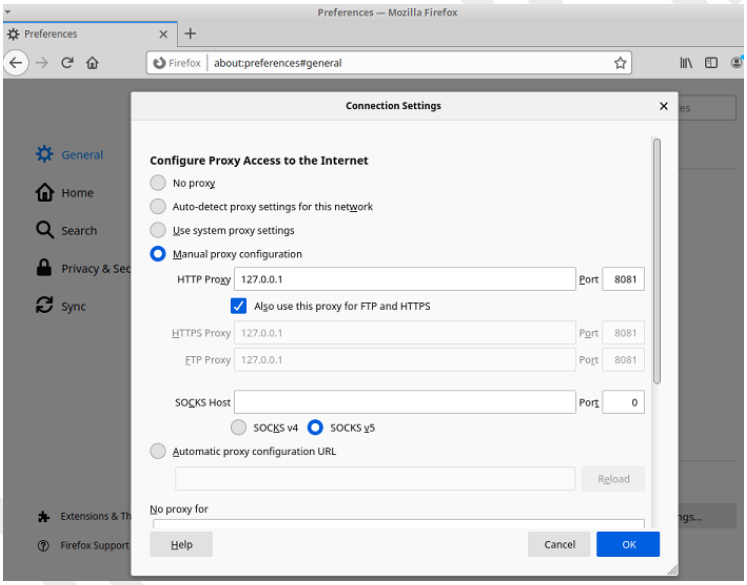



Figura 7: Configuración Proxy del navegador

En las conexiones HTTPS, para que el navegador confíe en la información que intercambia con el proxy (ZAP), es necesario añadir como CA el certificado que ZAP genera dinámicamente en cada instalación o cuando nosotros lo requiramos.

Paso 1: En las opciones de ZAP → Certificados SSL Dinámicos, descargamos el Certificado CA Raíz, que debe haberse creado al instalar ZAP.

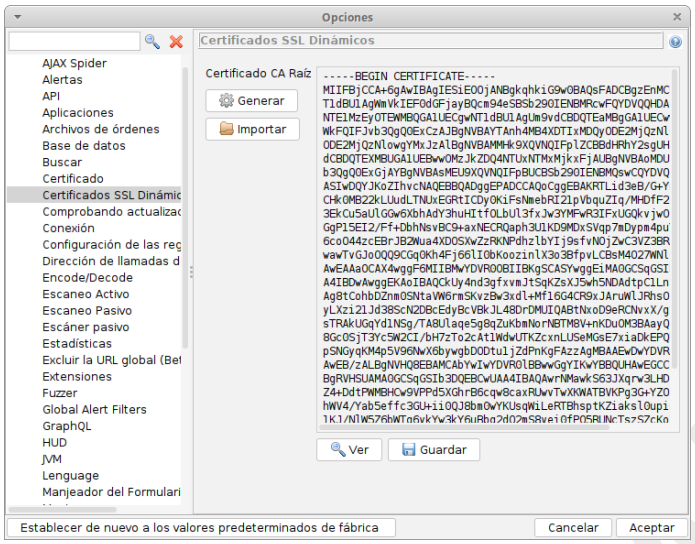


Figura 8: Configuración del certificado en OWASP ZAP

Paso 2: En las preferencias del navegador, debemos importar el Certificado Raíz de ZAP. Por ejemplo, para Mozilla Firefox, iremos a Privacy & Security → Certificate Manager → Authorities, e importamos el certificado que hemos descargado en el anterior paso. Debemos marcar al menos "Trust this CA to identify websites".

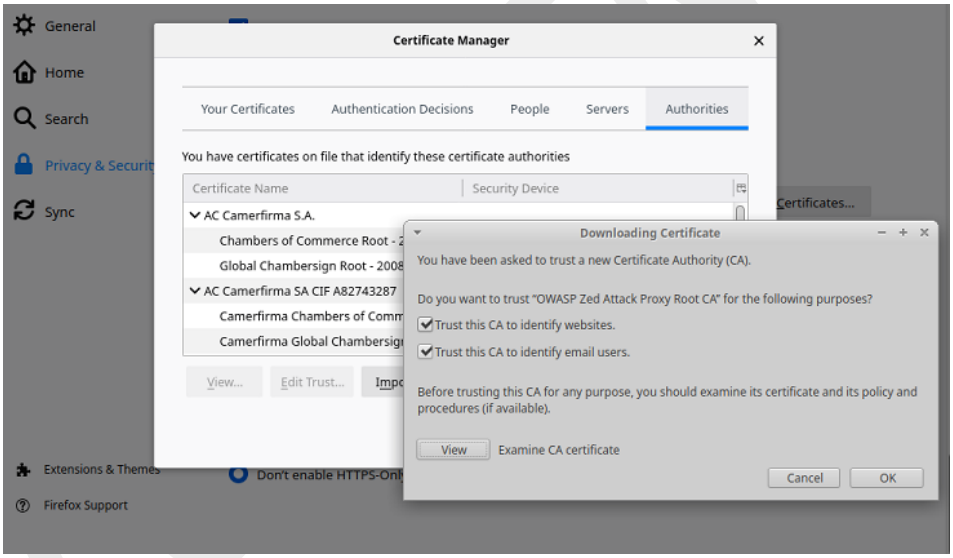


Figura 9: Importación y comprobación del certificado raíz.

Paso 3: Una vez importado debe aparecer en el apartado "Authorities". Si accedemos desde el navegador a una página usando HTTPS, al acceder al certificado de la página podremos observar que ZAP genera dinámicamente un certificado para cada URL, avalado por el certificado CA de ZAP. El navegador no mostrará mensajes de alerta ya que hemos incorporado el certificado CA de ZAP.

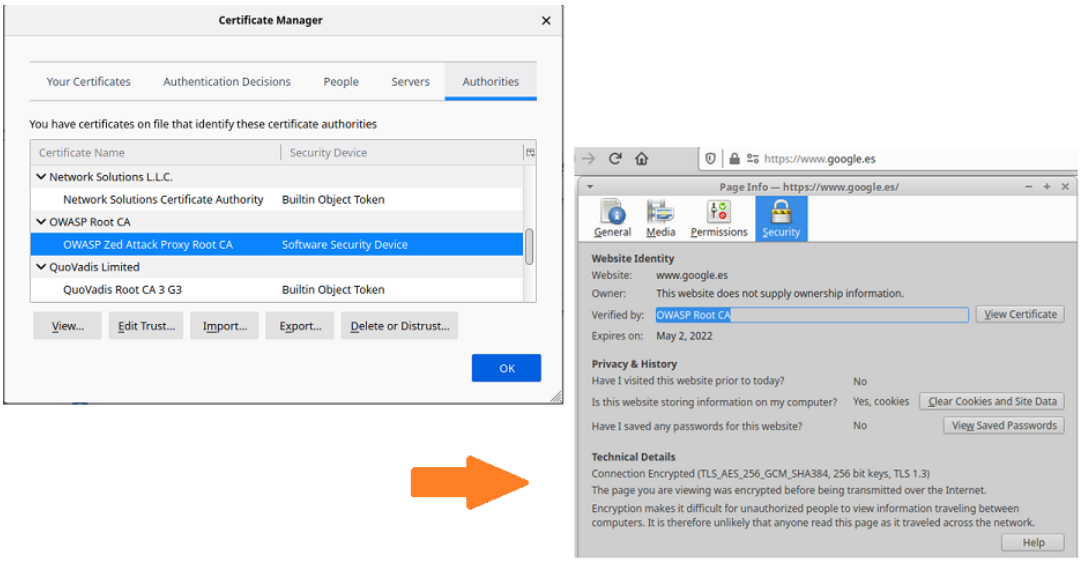



Figura 10: Comprobación de certificado en la conexión a través del proxy de OWAPS ZAP

Una vez realizada esta configuración en nuestro cliente web ("navegador"), las peticiones/respuestas HTTP/HTTPS pasarán a través del proxy de la herramienta OWASP ZAP y se podrán visualizar como se muestra a continuación en la figura 4. En esta ventana, también se puede además de visualizar la petición, modificar los datos de dicha petición y aceptar los cambios, cancelarlos, abortar la petición, y cancelar todas las interceptaciones.

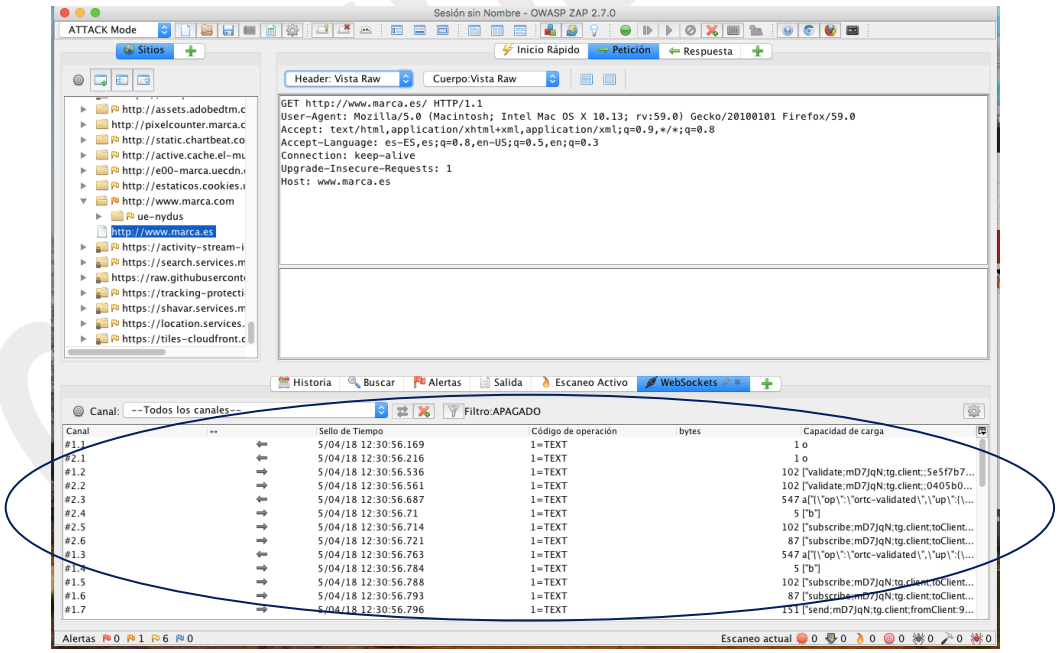



Figura 11: Ventana de interceptación de petición HTTP

## Trazabilidad del tráfico HTTP/HTTPS con OWASP ZAP

Si realizamos cualquier petición desde nuestro n web, ya desde OWASP ZAP podemos inspeccionar estas peticiones usando la pestaña History.

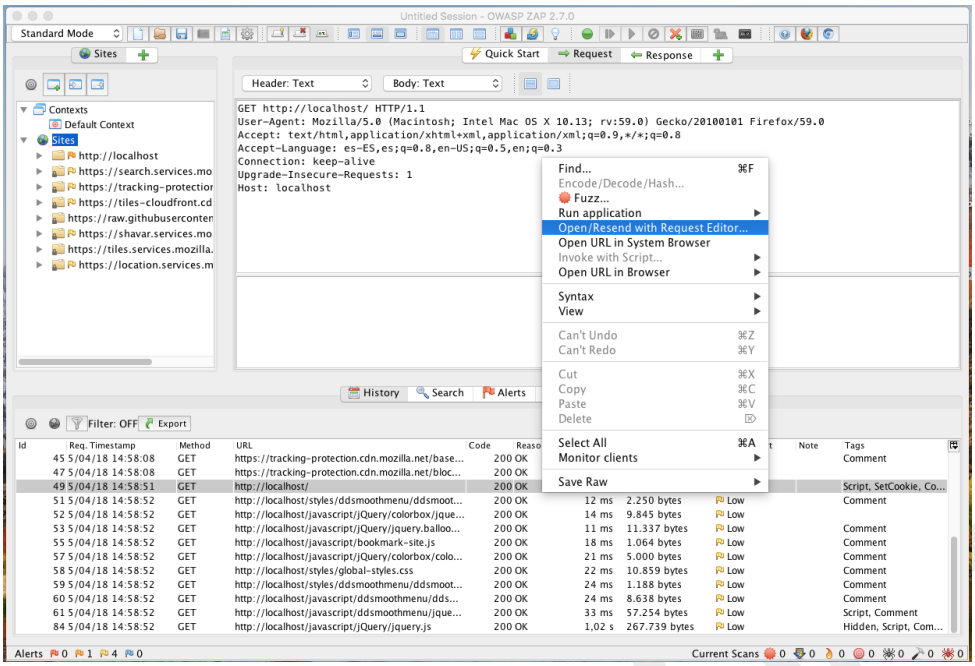



Figura 12: Visionado de peticiones

Haciendo doble-click en algunas de las peticiones realizadas se nos mostrará en la parte superior donde tenemos “Request”. Ya desde aquí podemos ver la información de una petición http y además podemos entrar en más detalles de esta incluso editarla. Desde la sección de “Request” con el botón derecho de ratón, en el menú contextual de “Open/Resend with Request Editor ... “ nos permitirá tener una vista más detallada de la petición incluso editarla.

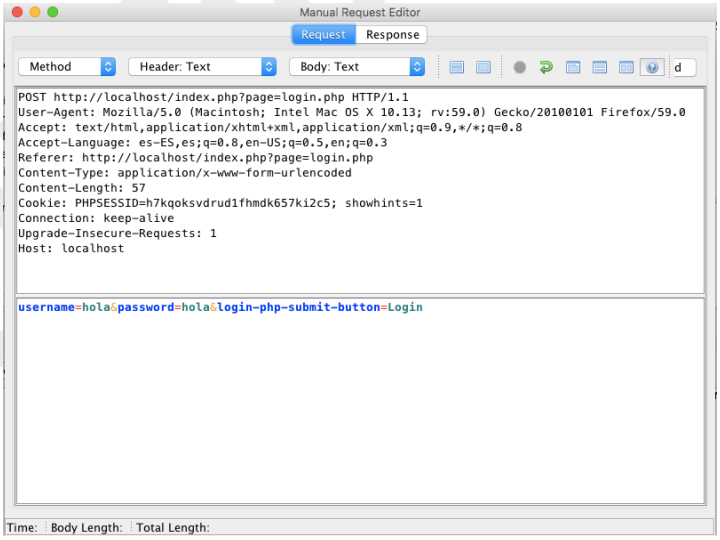



Figura 13: Ventana de inspección / edición de peticiones http.

En la figura anterior podemos ver dos partes:

- Primera parte: Información sobre la cabecera, en la figura podemos ver la cabecera de una petición POST para un login.
- Segunda parte: Información sobre los parámetros de la petición que se están enviando con la petición, en este caso login y password.

### Modo intercepción de peticiones con OWASP ZAP

Es posible activar un modo especial de ejecución donde capturaremos las peticiones y antes de ser respondidas por el servidor podremos visionar la petición incluso modificarla y reenviarla al servidor modificadas. Este es un caso de ejemplo de Man-in-the-Middle (MitM). Para activar el modo de intercepción sólo tenemos que activar un "Breakpoint", desde el botón verde situado en la barra de menús de ZAP.

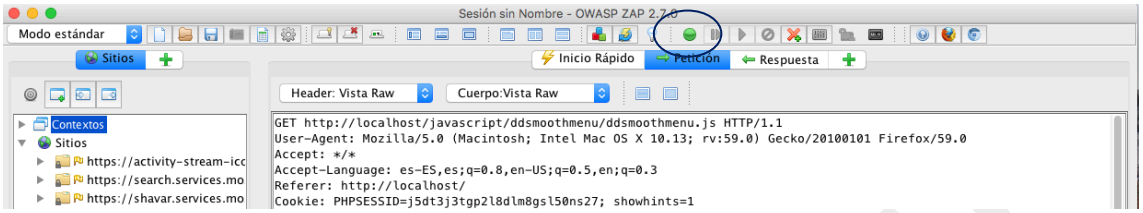



Figura 14: Breakpoint en OWASP ZAP

Una vez activada dicha opción ZAP se pondrá en modo recepción de peticiones y podremos capturar peticiones, en la Figura 15 se muestra un ejemplo de una petición de login hecha a través del proxy de ZAP puesto en modo intercepción.




Figura 15: Login de la aplicación web Mutillidae II de OWASP pasando por proxy de ZAP

CONFIDENCIAL

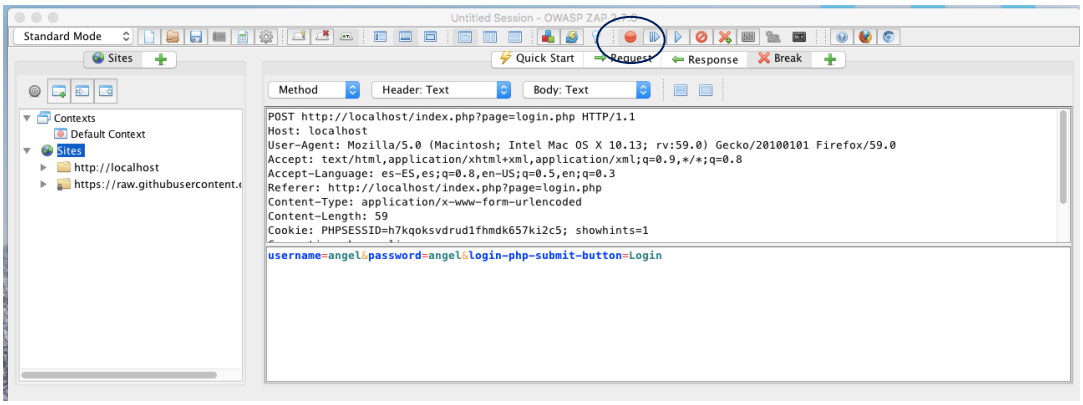



Figura 16: Captura de la petición http con los parámetros

En la parte de los parámetros podemos no sólo observar el nombre de las variables usadas en la aplicación, sino que también podemos ver el valor que toman dichas variables. Como se ha comentado anteriormente, esta ventana nos permite editar la petición y podríamos cambiar el valor de cualquiera de los parámetros de la petición insertando otros valores.

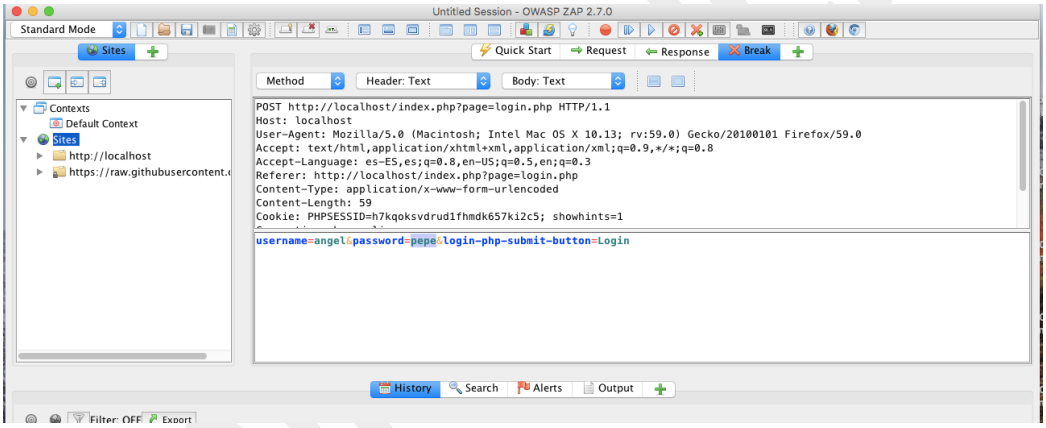



Figura 17: Petición modificada en el parámetro de password.

Una vez modificada la petición simplemente tendremos que darle salida a la petición pulsando en botón  en la barra de menús, y tendremos la petición de respuesta “Response”.

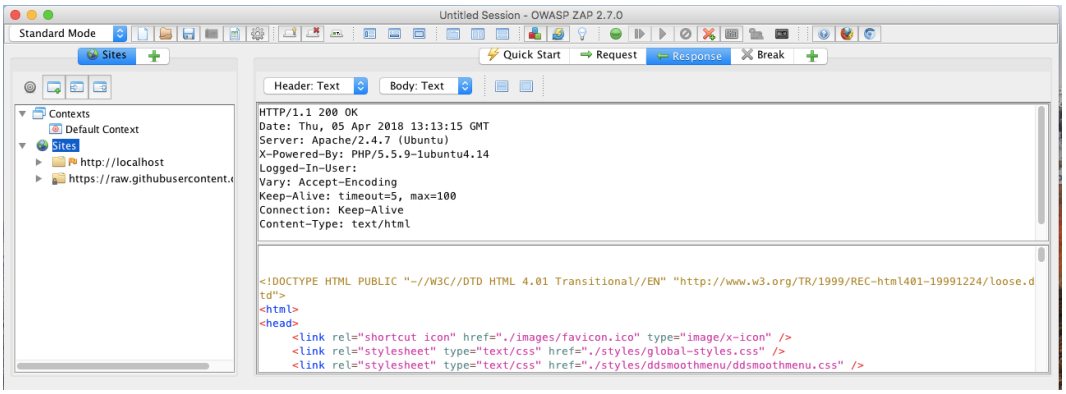


Figura 18: Petición de respuesta una vez cambiado los parámetros de la petición

A la vista de lo anteriormente expuesto, presente a INSEGUS un conjunto de pruebas que muestre las posibilidades de una de las herramientas seleccionadas de entre las propuestas para **comprobar la correcta trazabilidad de las peticiones HTTP/HTTPS sin modificación alguna de los parámetros de las peticiones** y las respuestas obtenidas desde el servidor de prueba sin realizar modificación alguna.

### Análisis de vulnerabilidades en aplicaciones web

Primeramente, reiterar a los ingenieros/as que el uso de esta herramienta para el análisis de vulnerabilidades se debe realizar siempre sobre aplicaciones web que no se encuentren en producción, por ejemplo, en aplicaciones en desarrollo propias, en entornos simulados (no reales) o de prueba de concepto. Cualquier otro tipo de actividad relacionada con el análisis de vulnerabilidades web que no sean sobre los anteriores entornos podría conllevar infracciones legales por el uso de esta herramienta, además de posibles repercusiones sociales y económicas difíciles de determinar. Por ello existen entornos web de pruebas con vulnerabilidades entre los que podemos encontrar: Gruyere, DVWA (Damn Vulnerable Web Application), WebGoat, VamPI, OWASP Juice Shop, TryHackMe, Hack The Box (HTB).

Entre las vulnerabilidades más típicas en aplicaciones Web se encuentran la inyección SQL, Path Traversal, XSS reflejado y almacenado (puede consultarse en OWASP Top Ten 2023). Estas vulnerabilidades habría que intentar mitigarlas o mejor evitarlas en los entornos web antes de ponerlos en producción ya que podrían permitir extraer información confidencial de las aplicaciones web y/o permitir ataques a los clientes, pues por ejemplo los campos en los que son introducidos determinados valores en los formularios no son validados correctamente por el servidor o enviados al cliente.

### Modo testeo de inyecciones con OWASP ZAP

Para realizar las pruebas de inyección no es necesario probar uno a uno las diferentes cadenas/payloads, ya que OWASP ZAP está provisto de un "Fuzzer" que nos permite una vez interceptada una petición, repetir la misma con diferentes parámetros de entrada. Para ello sobre cualquier petición "Request" que disponga de parámetros le damos botón derecho y tenemos la opción "Fuzz".

CONFIDENCIAL

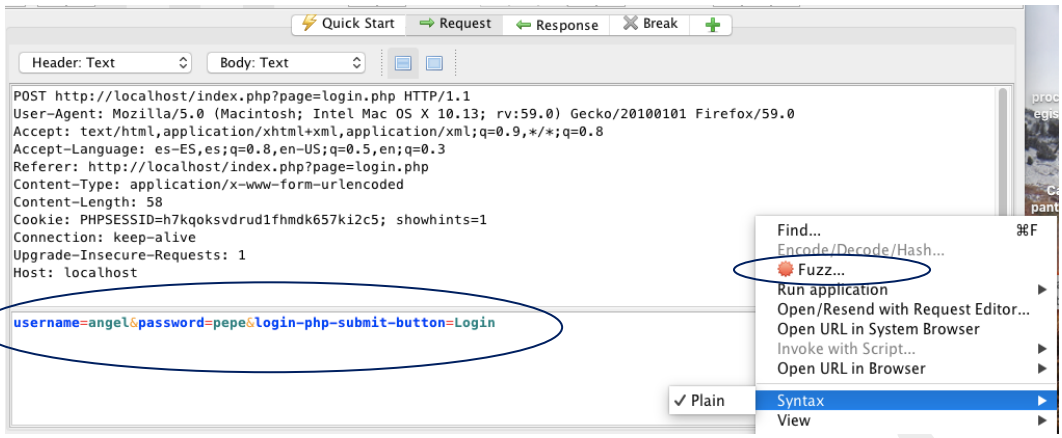



Figura 19: Acceso al Fuzzer desde una petición

Una vez pulsado en la opción de “Fuzz” ya tendremos acceso al Fuzzer que nos permitirá configurar que parámetros usaremos para introducir las diferentes cadenas de búsqueda para probar en las peticiones, y seguimos los siguientes pasos:

1. Seleccionar el parámetro donde introduciremos los payloads.
2. Pulsamos en Payloads ...
3. Seleccionamos el tipo, Type, File, y seleccionamos el archivo donde esté las cadenas.
4. Pulsamos Add, en la ventana de Add Payload
5. Pulsamos OK, en la ventanas de Payloads.
6. Start Fuzzer.

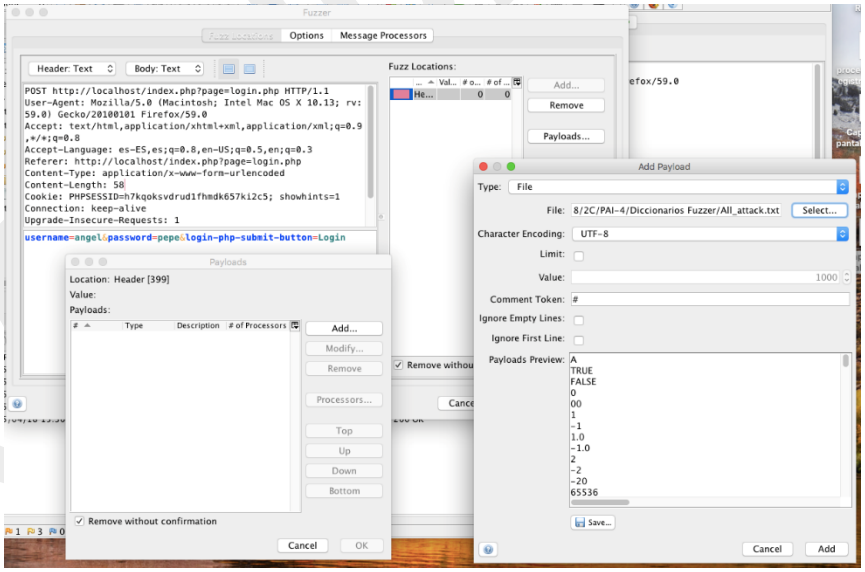



Figura 20: Configuración de los payloads

También ZAP viene provisto con un conjunto de diccionarios de payloads precargados para hacer pruebas de inyecciones. En este caso en lugar de marcar en el paso 3 que vamos a usar un fichero, marcamos la opción de “File Fuzzer”. Como podremos observar dentro de la opción de “jbrofuzz” tenemos una sección de “SQL Injection” y dentro tenemos varios diccionarios de payloads predefindos, podemos seleccionar alguno de estos para hacer las correspondientes pruebas.

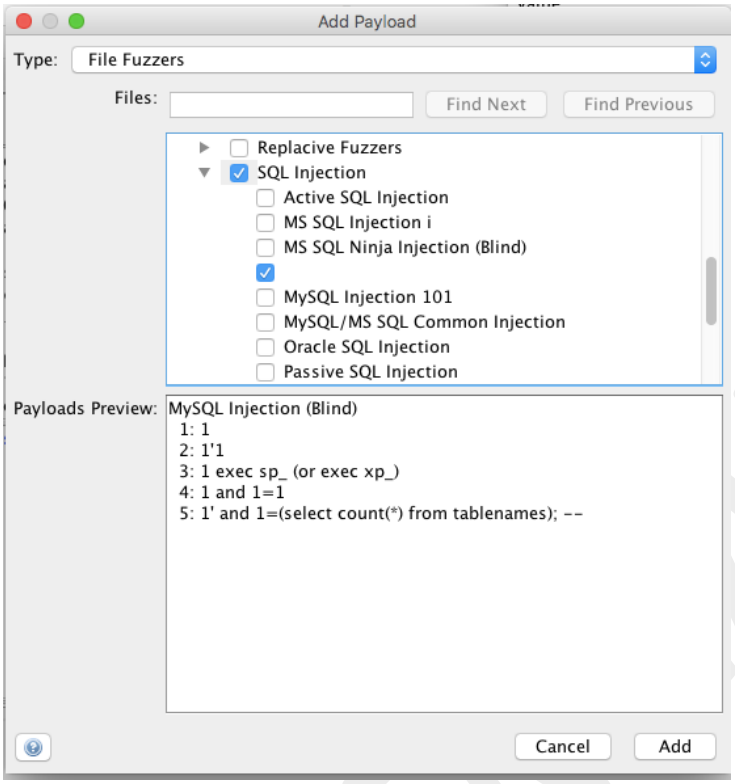



Figura 21: Lista de ficheros de payloads predefinidos en ZAP.

El Fuzzer se encargará de usar todas las cadenas del diccionario que hayamos introducido y tomar las respuestas, mostrándonos una pantalla de resultados. Será trabajo nuestro identificar qué peticiones han dado resultados satisfactorios y ver qué cadenas son las más interesantes. Todas aquellas peticiones marcadas como "Reflected" en su "Status" o como "Found" en su "Reason" son peticiones interesantes por sus resultados. Entre ellas podremos encontrar cosas interesantes.

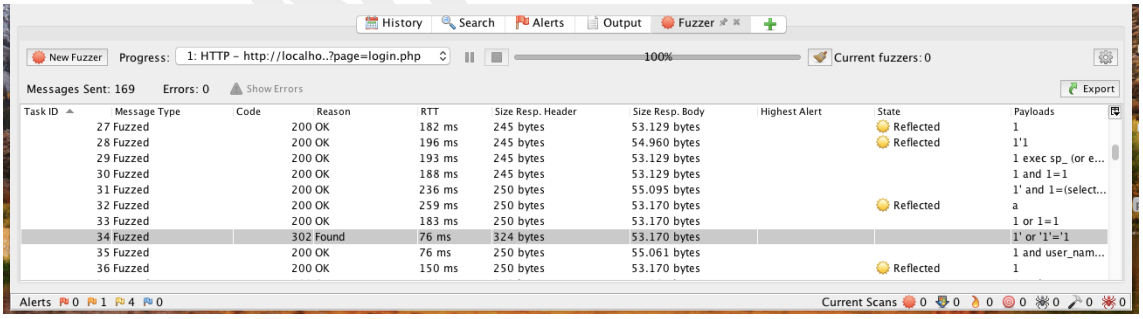



Figura 22: Resultados del Fuzzer

El objetivo será **seleccionar una aplicación web y realizar una auditoría para determinar potenciales vulnerabilidades**. Se deben presentar las pruebas que permitan determinar las potenciales vulnerabilidades explotables de al menos 3 de las recomendadas por OWASP Top 10. Además, se debe definir un plan de mitigación con las recomendaciones o los controles de seguridad que se deben desplegar para evitar las vulnerabilidades identificadas.

## Normas del entregable

- Cada Security Team debe entregar a través de la Plataforma de Enseñanza Virtual y en la actividad preparada para ello un archivo zip, nombrado PAI3-ST<Num>.zip, que deberá contener al menos los ficheros siguientes:
  - ✓ **Documento en formato pdf** que contenga un informe/resumen del proyecto con los detalles más importantes de las decisiones, soluciones adoptadas y/o implementaciones desarrolladas, así como el resultado y análisis de las pruebas realizadas (máximo 15 páginas). Estructura recomendada:
    - Portada e índice contenido
    - Resumen ejecutivo
    - Auditoría de sistemas informáticos
      - Descripción del sistema analizado y herramientas de análisis
      - Plan de pruebas y resultados
      - Conclusiones
    - Análisis y bastionado de navegador web
      - Descripción del sistema analizado
      - Plan de pruebas y resultados
      - Conclusiones
    - Auditoría de sistema móvil
      - Descripción del sistema analizado y herramientas de análisis
      - Plan de pruebas y resultados
      - Conclusiones
    - Auditoría de sistema web
      - Descripción del sistema web y herramientas de análisis
      - Plan de pruebas y resultados
      - Conclusiones
    - Referencias(no cuenta para el total de páginas permitidas)
    - Anexos (no cuenta para el total de páginas)

  - ✓ **Evidencias Técnicas:** Código fuente y/o scripts desarrollados y/o configuraciones utilizadas de las herramientas. Además, **será obligatorio la entrega de las pruebas/test realizados junto con los logs/trazas de todas y cada una de las herramientas de análisis utilizadas** (Lynis, ZAP o cualquier otra herramienta utilizada).
- El plazo de entrega de dicho proyecto finaliza el día 10 de noviembre a las 23:59 horas.
- Los proyectos entregados fuera del plazo establecido serán considerados inadecuados por el cliente y, por tanto, entrarán en penalización por cada día de retraso, con entrega de 10% del total, hasta agotarse los puntos.
- **El cliente no se aceptarán envíos realizados por email, ni mensajes internos de la enseñanza virtual, ni correo interno de la enseñanza virtual. Toda entrega realizada por estos medios conllevará una penalización del 10% en la entrega.**

CONFIDENCIAL